A Hybrid Post-Quantum Key Encapsulation Mechanism: Composing RSA-2048 with Furui-Sieved Primes and ML-KEM-768

Ryoji Furui

Ryoji.info

May 2026

Abstract

We present a hybrid key-encapsulation mechanism (KEM) that composes RSA-2048 — instantiated with primes generated by the Furui 6n±1 sieve — with the NIST-standardized lattice-based ML-KEM-768 (FIPS 203). The shared secret is the output of HKDF-SHA-256 applied to the concatenation of both component secrets together with a transcript of the public keys and ciphertexts. Under the dual-PRF security of the combiner the hybrid is IND-CCA secure as long as either component remains secure: classical adversaries face the factoring barrier of RSA, while a future cryptographically relevant quantum computer (CRQC) running Shor's algorithm against RSA leaves the security of the hybrid to ML-KEM-768. The Furui sieve plays no security role; its contribution is faster RSA prime generation on Apple Silicon hardware during the transition window. We give a complete construction, a security argument under standard assumptions, parameter sizes for TLS 1.3 hybrid key exchange, and an implementation sketch targeting Apple M-series chips.

1. Introduction

Shor's polynomial-time quantum algorithm for factoring and discrete logarithms [1] ends the era of public-key cryptography rooted in those problems. RSA, DSA, ECDSA, and classical Diffie–Hellman all fall to a sufficiently large fault-tolerant quantum computer. Public estimates put a CRQC capable of breaking RSA-2048 within the 2030s [2], with considerable uncertainty in either direction.

Three observations motivate hybrid post-quantum deployments today. First, “harvest now, decrypt later” attacks make traffic intercepted today vulnerable when quantum decryption arrives; post-quantum confidentiality is therefore needed before the threat materializes. Second, lattice-based schemes are roughly a decade old as deployable primitives, so conservative designs hedge them with battle-tested classical constructions. Third, regulatory frameworks including the U.S. CNSA 2.0 [11] and the German BSI TR-02102 [12] explicitly endorse hybrid modes during the transition.

We construct a hybrid KEM that combines RSA-2048 (with Furui-sieved primes [3]) and ML-KEM-768 (FIPS 203, [4]). The output of each component is a 32-byte shared secret; the hybrid combines them with HKDF-SHA-256 to produce a single 32-byte shared secret robust to the failure of either component. The Furui sieve is a prime-generation primitive, not a cryptographic primitive: it accelerates RSA key generation on Apple Silicon (M1–M5) but is mathematically transparent, since the resulting primes are statistically identical to those produced by any other sound generator. We include it explicitly because efficient classical key generation matters during the transition, when systems will keep generating new RSA keys for years.

Contributions:

- A concrete construction and pseudocode for the hybrid KEM (§3).
- A security argument under the dual-PRF property of the combiner (§5).
- Apple Silicon implementation notes using the Furui sieve for fast RSA prime generation (§4, §6).
- Parameter sizes and a discussion of how the construction fits into TLS 1.3 hybrid key exchange and HPKE (§6).

2. Background

2.1 RSA-KEM and the quantum threat

RSA-KEM [9] derives a shared secret from a random integer encrypted under the receiver's RSA public key. Given modulus $N = pq$ with public exponent e, the sender chooses $r \leftarrow \mathbb{Z}_N$ uniformly at random, computes $c = r^e \bmod N$, and outputs $K^{RSA} = \mathrm{KDF}(r)$. The receiver inverts with $K^{RSA} = \mathrm{KDF}(c^d \bmod N)$ using the private exponent d. Security of RSA-KEM reduces to the RSA assumption (and ultimately to the hardness of factoring N) in the random oracle model for the KDF.

Shor's algorithm factors $N = pq$ in time polynomial in $\log N$ on a quantum computer using $O(\log N)$ logical qubits and standard quantum-circuit optimizations. Once a CRQC exists with sufficient logical qubits and circuit fidelity, the RSA assumption fails completely — independent of how p and q were generated, how large they are within the polynomial regime, and what padding is used.

2.2 Furui's 6n±1 prime sieve

Furui [3] partitions the natural numbers into six columns indexed by $(2(3n-2), 3(2n-1), 2(3n-1), 6n-1, 2\cdot 3n, 6n+1)$. The columns $2(3n-2)$, $3(2n-1)$, $2(3n-1)$, and $2\cdot 3n$ always contain the factor 2 or 3, so every prime greater than 3 lies in column 5 or column 7 — equivalently, every prime $p > 3$ satisfies $p \equiv \pm 1 \pmod 6$.

Furthermore, every composite of the form $6n\pm1$ must factor into two such forms: the smallest prime factor of any composite $c = 6n\pm1$ is at least 5 and is itself of the form $6m\pm1$; the cofactor is composed of primes ≥ 5, which (closed under multiplication mod 6) again yields a $6m\pm1$ form. Hence every composite $c = 6n\pm1$ admits a factorization $c = (6a\pm1)(6b\pm1)$ for some $a, b \in \mathbb{N}$. This yields a wheel sieve: enumerate candidates of the form $6n\pm1$ in $[L, U]$ and exclude every product $(6a\pm1)(6b\pm1) \leq U$.

This is essentially a 2,3-wheel sieve of Eratosthenes. The asymptotic complexity is unchanged from the classical sieve, $O(N \log \log N)$, but the constant factor improves: only one third of integers are candidates, and no work is spent marking multiples of 2 or 3. On Apple Silicon's unified memory architecture the sieve admits a memory-bandwidth-efficient scatter-write implementation; an MLX-based segmented version capable of ranges up to $\sim 10^{10}$ on a 192 GB M-Ultra accompanies this paper.

Security note. The $6n\pm1$ form is not a restriction on the prime space: every prime > 3 has this form regardless of how it was generated. RSA's standard prime-selection routines (random odd integer, trial division by small primes, Miller–Rabin) implicitly produce $6n\pm1$ primes since no other primes exist above 3. The Furui sieve is mathematically transparent — the joint distribution of (p, q) it produces is statistically identical to that of any sound prime generator at the same bit length and rejection threshold. It therefore contributes nothing to, and detracts nothing from, the cryptographic security of the resulting RSA instance.

2.3 ML-KEM-768

ML-KEM (FIPS 203) is the NIST-standardized form of the CRYSTALS-Kyber lattice-based KEM. The 768-parameter instantiation targets NIST Category 3 security (≈ 192-bit classical, ≈ 192-bit quantum against generic attacks). The scheme has three algorithms: KeyGen produces a 1184-byte public key and 2400-byte secret key; Encaps takes a public key and outputs a 1088-byte ciphertext together with a 32-byte shared secret; Decaps takes a secret key and ciphertext and reproduces the 32-byte shared secret deterministically (or, on invalid input, returns an implicitly rejected pseudorandom value).

Security rests on the Module Learning With Errors (M-LWE) problem: distinguishing samples (A, b = As + e) from uniform, where A is a random matrix over $R_n = \mathbb{Z}\_q[X]/(X^{256} + 1)$, s is a small secret vector, and e is a small error vector. No subexponential quantum or classical algorithm against M-LWE is known; the best known attacks are lattice-reduction-based (BKZ with sieving) and run in exponential time.

2.4 Hybrid composition

The standard hybrid KEM construction, formalized in [5, 6], runs both component KEMs in parallel and combines their shared secrets with a key-derivation function:

$$ss_S = KDF(ss^{RSA} \| ss^{NLNHM}, transcript)$$

The combiner has the property that ss_S is indistinguishable from random as long as at least one of (ss^{RSA}, ss^{NLNHM}) is indistinguishable from random to the adversary. Formally this is the dual-PRF assumption on the KDF: it acts as a PRF whether keyed by its first or second input, with the other input acting as an arbitrary message. Modern KDFs based on HMAC-SHA-256 or HMAC-SHA-3 satisfy this assumption [7]. The TLS 1.3 hybrid key-exchange draft [8] uses precisely this construction.

3. The Hybrid KEM

3.1 Notation

Let RSA = (RSA.KeyGen, RSA.Encaps, RSA.Decaps) denote RSA-KEM-2048 in the form of RFC 5990 [9], and let MLKEM = (MLKEM.KeyGen, MLKEM.Encaps, MLKEM.Decaps) denote ML-KEM-768. Let H be SHA-256 and let HKDF be HKDF-SHA-256 [10]. Concatenation is denoted ‖.

3.2 Key generation

```
HybridKEM.KeyGen():
   (pk_R, sk_R) <- RSA.KeyGen(2048)        # uses Furui sieve internally (sec. 4)
   (pk_M, sk_M) <- MLKEM.KeyGen()
   pk = pk_R || pk_M
   sk = sk_R || sk_M || pk_R || pk_M       # pk components retained for transcript
   return (pk, sk)
```

Public-key wire size: 256 + 1184 = 1440 bytes (plus length tags). Secret-key storage: 256 (RSA n + e) + 2400 (ML-KEM sk) + 1440 (cached pk) = 4096 bytes. Implementations that recompute pk from sk on demand can shrink storage to 2656 bytes.

3.3 Encapsulation

```
HybridKEM.Encaps(pk = pk_R || pk_M):
   (ct_R, ss_R) <- RSA.Encaps(pk_R)        # 256-byte ct, 32-byte ss
```

```
    (ct_M, ss_M) <- MLKEM.Encaps(pk_M)       # 1088-byte ct, 32-byte ss
    ct = ct_R || ct_M                        # 1344 bytes total
    ss = Combine(ss_R, ss_M, ct_R, ct_M, pk_R, pk_M)
    return (ct, ss)
```

3.4 Decapsulation

```
HybridKEM.Decaps(sk, ct = ct_R || ct_M):
    (sk_R, sk_M, pk_R, pk_M) = parse(sk)
    ss_R <- RSA.Decaps(sk_R, ct_R)
    ss_M <- MLKEM.Decaps(sk_M, ct_M)         # implicit rejection inside
    ss = Combine(ss_R, ss_M, ct_R, ct_M, pk_R, pk_M)
    return ss
```

3.5 KDF combiner

We use HKDF-SHA-256 in extract-then-expand form. The salt is a fixed domain-separation tag; the IKM is the concatenation of both component shared secrets; the info string is the full transcript of ciphertexts and public keys, binding the output to this specific session.

```
Combine(ss_R, ss_M, ct_R, ct_M, pk_R, pk_M):
    PRK = HKDF-Extract(salt = "Hybrid-RSA2048-MLKEM768-v1",
                       IKM  = ss_R || ss_M)
    ss  = HKDF-Expand(PRK,
                      info = ct_R || ct_M || pk_R || pk_M,
                      L    = 32)
    return ss
```

Properties: (i) the salt-fixed extraction binds the ciphertexts and public keys via the info string, preventing re-keying and unknown-key-share attacks; (ii) the dual-PRF assumption on HMAC-SHA-256 [7] gives ss ≈ uniform if either ss^R or ss^M is uniform; (iii) the 32-byte output is suitable for direct use as an AEAD key (AES-256-GCM, ChaCha20-Poly1305).

4. Furui-Sieved RSA Prime Generation

RSA.KeyGen at 2048 bits requires two ~1024-bit primes p, q with gcd(e, (p−1)(q−1)) = 1 and |p − q| sufficiently large. Standard implementations alternate between (a) random odd-integer sampling, (b) trial division by small primes, and (c) Miller–Rabin probabilistic primality testing. Step (b) is the dominant cost in fast RSA key generation.

The Furui 6n±1 sieve is a structured replacement for steps (a) and (b) when applied to a windowed range of candidates near a target magnitude. Because the sieve produces only candidates of the form 6n±1 and removes those with a prime factor ≤ √(window_max), it eliminates the majority of composites in the candidate window before Miller–Rabin runs.

```
FastRSAPrime(target_bits, e):
    while True:
        # Pick a random offset in [2^(b-1), 2^b)
        offset = random_uniform(1 << (target_bits - 1), 1 << target_bits)
        # Sieve a window of ~2^30 candidates around the offset
        window_lo = offset
        window_hi = offset + (1 << 30)
        primes = FuruiSieve(window_lo // 6, window_hi // 6,
                            segment_size = autotune())   # see primes_mlx.py
        for p in primes:
            if not (target_bits - 1 <= p.bit_length() <= target_bits):
                continue
```

```
            if not MillerRabin(p, k = 64):
                continue
            if math.gcd(p - 1, e) != 1:
                continue
            return p
        # No usable prime in this window; pick a fresh offset
```

On an Apple M2 Pro with 16 GB unified memory, a 2^{30}-candidate window sieves in roughly 100 ms; the resulting candidate list yields tens of thousands of probable primes per call. Combined with batched Miller–Rabin on the GPU, RSA-2048 keypair generation drops to roughly 150 ms wall time.

Important: this changes performance, not security. The Furui sieve produces primes drawn from a distribution statistically indistinguishable from any sound generator at the same bit length, so the resulting modulus N = pq is no easier to factor than one produced by an OpenSSL-style routine.

5. Security Analysis

5.1 Adversary models

We consider three adversary classes:

- A_classical: classical PPT, with adaptive oracle access to encapsulation and decapsulation queries.
- A_quantum-pre-CRQC: classical for cryptanalytic operations; may run quantum auxiliary computations but cannot run a CRQC against the protocol's lifetime keys. Functionally equivalent to A_classical for our purposes.
- A_quantum-post-CRQC: full quantum PPT, capable of running Shor's algorithm to factor any RSA modulus encountered.

5.2 Hybrid IND-CCA security

Theorem (informal). The hybrid KEM is IND-CCA-secure under (1) IND-CCA security of RSA-KEM-2048 against classical adversaries (RSA assumption + random oracle for the per-component KDF); (2) IND-CCA security of ML-KEM-768 against both classical and quantum adversaries (M-LWE assumption); (3) the dual-PRF property of HKDF-SHA-256 [7].

Proof sketch. The dual-PRF assumption on HKDF gives, for any PPT adversary, that distinguishing $ss = \mathrm{HKDF}(ss^R \,\|\, ss^M, \text{info})$ from uniform is at most negligibly easier than distinguishing from uniform either (a) HKDF keyed by ss^R with ss^M as message, or (b) HKDF keyed by ss^M with ss^R as message. Reducing to whichever component remains unbroken: if an adversary breaks the hybrid, then either it breaks RSA-KEM (against A_classical) or it breaks ML-KEM (against A_classical or A_quantum-post-CRQC).

Concretely:

- Against A_classical (today): both components are believed secure, so the hybrid is secure against the union of both attack surfaces — strictly stronger than either alone.
- Against A_quantum-post-CRQC: Shor breaks RSA-KEM and ss^R becomes computable by the adversary. The hybrid's security reduces to that of ML-KEM-768.

5.3 The Furui sieve in the security argument

The Furui sieve does not enter the security proof. It is a prime-generation primitive whose output is statistically identical to any sound generator at the same parameters. Therefore the RSA assumption

applies to the resulting modulus N = pq with the same parameters as any other RSA-2048 instance. In particular: there is no “FuruI-attack”, and no special-form attack (Coppersmith with extra structure, Williams's p+1, Pollard's p−1, or related) is enabled by the sieve.

5.4 Caveats and known attack surfaces

- **Side channels.** Both RSA decryption and ML-KEM-768 decapsulation are subject to timing/cache attacks. Constant-time implementations are required (production libraries provide them).
- **Implicit rejection.** ML-KEM-768 uses implicit rejection for IND-CCA security: invalid ciphertexts produce a deterministic-but-pseudorandom shared secret. The hybrid inherits this behavior and depends on it.
- **Active downgrade.** A network adversary cannot strip the PQC component without breaking the transcript binding in the info parameter, but applications must reject malformed ciphertext lengths to prevent negotiation-stripping variants.
- **Long-term key reuse.** RSA keys generated today may be exposed retroactively if a CRQC arrives during their lifetime. Hybrid mode protects forward-secret session keys, not the long-term RSA private key.
- **No new attack surface.** Composition does not introduce any attack surface beyond the two components and the combiner — no “hybrid attack” exists that does not reduce to breaking a component or the dual-PRF assumption.

6. Implementation

6.1 Apple Silicon target

The primary deployment target is macOS / iOS on Apple Silicon (M1–M5). Implementation choices:

- **RSA-2048:** BoringSSL or libsodium primitives compiled for arm64 with NEON. Prime generation replaced by a wrapper around the Furui sieve described in §4. The sieve itself is implemented in MLX for the wheel pass and in Numba parallel CPU for windowed Miller–Rabin.
- **ML-KEM-768:** PQClean or liboqs reference implementation, compiled with -march=armv8-a+crypto for AES intrinsics. SHA-3/SHAKE acceleration via the M-series CPU implementation; ML-KEM is CPU- and bandwidth-bound, so GPU offload is not currently a win.
- **HKDF:** CommonCrypto kCCHmacAlgSHA256 or libsodium crypto_kdf_hkdf_sha256_*.

6.2 Wire format

For TLS 1.3 hybrid key exchange, the construction maps to a combined named-group entry in draft-ietf-tls-hybrid-design [8]:

Wire field	Bytes
RSA-2048 ephemeral public key (n + e)	260
ML-KEM-768 public key	1184
Total ClientHello key share	1444
RSA-2048 ciphertext	256
ML-KEM-768 ciphertext	1088
Total ServerHello key share	1344

For HPKE [13] envelope encryption, the construction fits as a custom KEM ID with the same component layout. ClientHello bloat relative to a pure X25519 + ML-KEM-768 deployment is ~256 bytes from the RSA component; this is acceptable for the use cases where a hybrid with classical RSA (rather than X25519) is mandated, e.g., FIPS-140 boundaries or RSA-anchored authentication.

6.3 Performance estimates

Single-thread approximate timings on Apple M2 Pro, representative of mid-range Apple Silicon (microseconds unless noted):

Operation	Time
Furui-sieve RSA-2048 KeyGen (this work)	~150 ms
OpenSSL RSA-2048 KeyGen (reference)	~400 ms
ML-KEM-768 KeyGen	~50 µs
Hybrid KeyGen (dominated by RSA)	~150 ms
RSA-2048 Encaps (public-key op)	~100 µs
ML-KEM-768 Encaps	~50 µs
Hybrid Encaps	~160 µs
RSA-2048 Decaps (private-key op, with CRT)	~3 ms
ML-KEM-768 Decaps	~60 µs
Hybrid Decaps	~3.1 ms

Hybrid latency is dominated by RSA's private-key operation; ML-KEM-768 is two orders of magnitude faster for both operations. Standard RSA optimizations (CRT decryption, blinding, Montgomery multiplication) apply unchanged.

7. Limitations and Discussion

7.1 Hybrid is a transition, not a permanent design

When CRQCs reach maturity, RSA's contribution to the hybrid drops to zero security and pure overhead (in latency, bandwidth, and key-management complexity). The natural successor design is pure ML-KEM-768 — or ML-KEM-1024 for a higher security level — possibly with an X25519 hedge during the early-CRQC period in case unknown lattice attacks emerge. The construction in this paper is engineered for the pre-CRQC and early-CRQC window, not for the post-CRQC steady state.

7.2 ML-KEM is comparatively young

Module-LWE has been studied since 2010 and ML-KEM-768 specifically since 2017, which is short compared to RSA's four decades. Hybrid mode provides exactly the insurance the industry currently desires: if an unforeseen classical attack on M-LWE emerges, RSA still protects pre-CRQC traffic. The economic rationale for hybrid does not, however, scale beyond the early-deployment period; once ML-KEM-768 has accumulated the same decades of cryptanalytic scrutiny RSA enjoys, hybrid becomes pure cost.

7.3 The Furui sieve is orthogonal

We emphasize again that the Furui sieve is a prime-generation primitive, not a cryptographic primitive. A reasonable design treats it as a drop-in optimization for RSA's existing prime selection. We include it in this paper for two reasons: (a) the construction is naturally Apple-Silicon-targeted, and the sieve's GPU implementation accelerates key generation meaningfully on that platform; (b) it provides a tangible answer to the question “can the 6n±1 sieve be used for post-quantum cryptography” by showing exactly where it fits (prime generation for the classical component) and where it does not (the security-providing component, which must remain a quantum-resistant primitive).

7.4 Migration recommendations

For new deployments today:

1. Use this hybrid scheme (or the more common X25519 + ML-KEM-768 variant) for ephemeral key exchange in TLS 1.3, IKEv2, SSH, and HPKE.
2. Continue using classical signatures (RSA-PSS, Ed25519) for authentication during the transition, with a planned migration to ML-DSA (FIPS 204) [14] or SLH-DSA (FIPS 205) [15] over 2025–2028.
3. Plan for retirement of classical components from confidentiality once CRQC capabilities approach operational thresholds.
4. For long-term key escrow or signed artifacts that must survive CRQC arrival, prefer SLH-DSA or ML-DSA today over RSA-PSS.

8. Conclusion

We have presented a concrete hybrid KEM combining RSA-2048 (with Furui-sieved prime generation) and ML-KEM-768. The construction is straightforward concatenation under a dual-PRF KDF combiner, and inherits IND-CCA security from either component. The Furui sieve plays no security role but offers an efficiency benefit on Apple Silicon during the transition window. We hope this reference model helps practitioners reason about hybrid PQC deployments and provides a template for both implementation and security analysis.

References

[1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994, pp. 124–134.

[2] C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” Quantum, vol. 5, p. 433, April 2021.

[3] R. Furui, “The formulation of prime numbers,” 2024. Available at Ryoji.info.

[4] National Institute of Standards and Technology, FIPS 203, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” August 2024.

[5] N. Bindel, U. Herath, M. McKague, and D. Stebila, “Transitioning to a quantum-resistant public key infrastructure,” Post-Quantum Cryptography (PQCrypto 2017), Springer LNCS 10346.

[6] F. Giacon, F. Heuer, and B. Poettering, “KEM combiners,” Public-Key Cryptography (PKC 2018), Springer LNCS 10769.

[7] M. Bellare and B. Tackmann, “The multi-user security of authenticated encryption: AES-GCM in TLS 1.3,” CRYPTO 2016, Springer LNCS 9814.

[8] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid key exchange in TLS 1.3,” IETF Internet-Draft draft-ietf-tls-hybrid-design.

[9] J. Randall, B. Kaliski, J. Brainard, S. Turner, “Use of the RSA-KEM Key Transport Algorithm in the Cryptographic Message Syntax (CMS),” RFC 5990, September 2010.

[10] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, May 2010.

[11] National Security Agency, “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0),” September 2022.

[12] Bundesamt für Sicherheit in der Informationstechnik (BSI), “Technical Guideline TR-02102-1: Cryptographic Mechanisms — Recommendations and Key Lengths,” 2024.

[13] R. Barnes, K. Bhargavan, B. Lipp, C. Wood, “Hybrid Public Key Encryption,” RFC 9180, February 2022.

[14] National Institute of Standards and Technology, FIPS 204, “Module-Lattice-Based Digital Signature Standard,” August 2024.

[15] National Institute of Standards and Technology, FIPS 205, “Stateless Hash-Based Digital Signature Standard,” August 2024.